

CI/CD Observability in Containerized Environments

Eduardo Sousa

**A dissertation proposal work plan for the degree of
of Master of Science, Specialisation Area of Cybersecurity And
Systems Administration**

**Advisor: Dr. Alexandre Bragança
Supervisor: Dr. Joao Silva**

Statement of Integrity

I hereby declare having conducted this academic work with integrity.

I have not plagiarised or applied any form of undue use of information or falsification of results along the process leading to its elaboration.

Therefore the work presented in this document is original and authored by me, having not previously been used for any other end.

I further declare that I have fully acknowledged the Code of Ethical Conduct of P.PORTO.

ISEP, Porto, January 6, 2026

Abstract

Internal CI/CD services (GitLab, Jenkins, Nexus, SonarQube, Dependency-Track) often run in Docker on mixed Linux/Windows hosts without administrative access. In these constrained environments, teams struggle to detect faults quickly and to diagnose root causes because telemetry is fragmented and host-level instrumentation is not permitted. This work proposes a container-first observability architecture that collects metrics, logs and traces from non-privileged containers using sidecar exporters, agent collectors and runtime APIs, and correlates signals in a unified metrics pipeline with dashboards and alerts. The approach is validated on a four-server, on-premise setup and evaluated with MTTD, MTTR, pipeline throughput, alert precision and overhead (target below 5%). The expected contribution is a reference architecture, reproducible prototype and decision guidance that improve CI/CD reliability under strict privilege constraints. **Keywords:** CI/CD, Observability, Containers,

Tracing, Logging, Monitoring

Resumo

Trabalhos escritos em língua Inglesa devem incluir um resumo alargado com cerca de 1000 palavras, ou duas páginas.

Se o trabalho estiver escrito em Português, este resumo deveria ser em língua Inglesa, com cerca de 200 palavras, ou uma página.

Para alterar a língua basta ir às configurações do documento no ficheiro `main.tex` e alterar para a língua desejada ('english' ou 'portuguese')¹. Isto fará com que os cabeçalhos incluídos no template sejam traduzidos para a respetiva língua.

¹Alterar a língua requer apagar alguns ficheiros temporários; O target **clean** do **Makefile** incluído pode ser utilizado para este propósito.

Contents

List of Figures	xi
List of Tables	xiii
List of Symbols	xv
List of Acronyms	xvii
1 Introduction	1
1.1 Introduction and Problem Framing	1
1.2 Context and Motivation	2
1.2.1 Theoretical Context	2
1.2.2 Organizational Reality	2
1.3 Problem Statement	3
1.3.1 Technical Problem	3
1.3.2 Organizational Problem	3
1.3.3 Scope and Boundaries	4
1.4 Intended Contributions	4
1.5 Research Objectives	4
1.6 Research Questions	5
1.7 Methodology Overview	5
1.8 Ethical Considerations	6
1.9 Document Structure	7
2 Literature review	9
2.1 PRISMA Methodology Overview	9
2.1.1 Identification	10
2.1.2 Screening	10
2.1.3 Eligibility	10
2.1.4 Inclusion and Data Extraction	10
2.2 PRISMA Flow Diagram	11
2.3 Instrumentation and Deployment Assumptions in the Literature	12
2.4 Performance Evaluation Practices and Reported Overheads	13
2.5 Discrepancies Between Research Environments and Constrained CI/CD Contexts	14
2.6 Practitioner-Oriented Approaches and Industry Practices	15
2.7 Identified Gaps and Implications for Constrained CI/CD Observability	16
2.8 Mapping of Research Questions to Thesis Chapters	17
2.8.1 RQ1: Non-Privileged Telemetry Collection	17
2.8.2 RQ2: Cross-Server Correlation and Performance Trade-offs	17
2.8.3 RQ3: Constraints, Trade-offs and Mitigation Strategies	18

2.9	Summary and Implications for Subsequent Research	19
3	Project Planning and Methodology	21
3.1	Objectives and Scope Alignment	21
3.2	Methodological Approach	21
3.2.1	Planned Artifacts	21
3.2.2	Development Process and Traceability	22
3.2.3	Planned Evaluation Strategy	22
3.3	Skills Development Plan (PREPD)	23
3.3.1	Competency Diagnosis	23
3.4	Project Planning and Schedule	23
3.4.1	Phases, Activities, and Deliverables	23
3.4.2	Milestones and Dependencies	23
3.4.3	Gantt Chart	25
3.5	Project Monitoring and Management	25
3.6	Risk Management	26
4	Conclusion and Next Steps	29
4.1	Synthesis of Preparation Work	29
4.2	Transition to Thesis Execution	29
	Bibliography	31

List of Figures

2.1	PRISMA flow diagram illustrating the systematic literature review process [3]. The diagram shows the progression from initial identification through screening, eligibility assessment, and final inclusion stages, documenting the number of records at each stage and reasons for exclusion.	11
3.1	Project schedule overview (Gantt chart; January–June 2026)	25

List of Tables

3.1	Competency Diagnosis and Targets	23
3.2	Skills Development Actions Mapped to Project Phases	24
3.3	Planned Phases, Activities, and Deliverables (January–June 2026)	24
3.4	Planned Milestones and Acceptance Focus	25
3.5	Risk Rating Scales and Measurement Approach	26
3.6	Risk Register (technical, schedule, and external risks)	27

List of Symbols

a	distance	m
P	power	W (Js^{-1})
ω	angular frequency	rad

List of Acronyms

CI/CD	Continuous Integration / Continuous Delivery.
eBPF	extended Berkeley Packet Filter.
MTTD	Mean Time To Detection.
MTTR	Mean Time To Recovery.
PREPD	Preparation of Dissertation.
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses.

Chapter 1

Introduction

1.1 Introduction and Problem Framing

Infrastructure failures in containerized Continuous Integration / Continuous Delivery (CI/CD) platforms create immediate operational challenges: determining which service failed, identifying the root cause, and restoring functionality before development teams are blocked. In organizations running onpremise CI/CD infrastructure, this diagnostic process often requires manual inspection of logs across multiple servers, correlation of timestamps between disconnected systems, and inference from incomplete information. This fragmentation extends detection and resolution times, directly affecting development velocity and release schedules.

The problem arises from a mismatch between observability tooling assumptions and operational realities. Standard monitoring solutions assume either full administrative access to host systems or cloudmanaged platforms with integrated telemetry collection. Organizations managing containerized infrastructure without host administrative privileges fit neither model. They cannot deploy hostlevel monitoring agents, cannot execute privileged containers due to security policies, and cannot modify container images of thirdparty services such as GitLab, Jenkins or SonarQube. Despite these constraints, operational visibility into service health, performance and fault conditions remains essential.

This project investigates observability approaches for containerized CI/CD platforms operating under nonprivileged access constraints and heterogeneous infrastructure. The work examines patterns for collecting, correlating and analyzing telemetry (metrics, logs, traces) when conventional monitoring deployment models are unavailable. The investigation focuses on a fourserver onpremise deployment running mixed Linux and Windows hosts, where operations teams lack host administrative access and must instrument services without modifying container images. The objective is to evaluate whether containerfirst instrumentation patterns can measurably reduce fault detection and diagnosis time while maintaining resource overhead below operationally acceptable thresholds. This preparation report documents the planned research strategy, including systematic literature review, technology assessment and empirical evaluation methodology.

The central research problem is: *How can an integrated observability framework be designed and evaluated for containerized CI/CD platforms operating under nonprivileged access constraints and heterogeneous infrastructure, to assess its potential to improve reliability and reduce incident response times?*

1.2 Context and Motivation

1.2.1 Theoretical Context

Continuous integration and continuous delivery (CI/CD) pipelines constitute critical infrastructure for software development organizations. Research in software engineering has established that pipeline reliability and observability directly influence development velocity, defect detection rates and timetomarket [1]. Modern CI/CD platforms typically comprise multiple specialized services: source control management (GitLab), build automation (Jenkins), static analysis (SonarQube), artifact management (Nexus) and security scanning (DependencyTrack). Each service generates distinct telemetry streams (metrics, logs, traces) that must be collected, correlated and analyzed to maintain operational visibility.

Observability frameworks in distributed systems follow established patterns: metrics collection for quantitative monitoring, structured logging for event analysis and distributed tracing for transaction flow reconstruction. Cloudnative observability assumes elastic infrastructure, privileged access to host resources and homogeneous execution environments. These assumptions enable instrumentation patterns such as node exporters with host access, kernellevel tracing and infrastructureascode deployment of monitoring agents.

1.2.2 Organizational Reality

The organization where this project is developed manages onpremise CI/CD infrastructure serving multiple development teams across different business units. The platform runs on four physical servers: three Linux hosts and one Windows Server 2019 host. All CI/CD services operate as Docker containers to facilitate deployment and isolation.

Two organizational constraints fundamentally shape the problem space. First, operations teams do not possess administrative privileges on host systems. Infrastructure administration is centralized under a separate IT department following corporate security policies. This restriction prevents installation of hostlevel monitoring agents, kernel modules or systemwide instrumentation. Second, CI/CD service containers must run without elevated privileges. Corporate security policy prohibits privileged container execution due to isolation and attack surface concerns.

These constraints create a gap between operational needs and available deployment options. Many standard monitoring approaches require capabilities that are unavailable in this environment: hostlevel access for system metrics, privileged container execution for kernel instrumentation, or the ability to modify service container images. The organization currently relies on applicationlevel logs accessed through Docker CLI commands, manual inspection of container resource usage and reactive troubleshooting after incidents are reported by end users. This reactive posture results in prolonged incident detection (often measured in hours rather than minutes) and diagnosis cycles that require coordinated investigation across multiple disconnected log sources.

The practical impact manifests in three measurable ways: delayed detection of service degradation until user complaints escalate, extended root cause analysis requiring manual correlation of logs from five or more services, and inability to establish proactive capacity planning due to lack of historical resource utilization data. Development teams experience unpredictable pipeline failures attributed to infrastructure issues that operations teams cannot rapidly diagnose or prevent.

1.3 Problem Statement

1.3.1 Technical Problem

Containerized CI/CD platforms deployed onpremise with nonprivileged container execution and mixed operating system environments present observability challenges that require careful investigation. The technical problem comprises three interconnected dimensions:

1. **Instrumentation constraints:** Nonprivileged containers cannot access hostlevel metrics (CPU, memory, disk, network at host scope), kernel instrumentation or systemwide performance counters. Containervisible metrics provide limited visibility into resource contention, I/O bottlenecks or hostlevel saturation.
2. **Telemetry fragmentation:** Each CI/CD service generates independent log streams in different formats (plain text, JSON, structured). Without centralized collection and correlation, incident diagnosis requires sequential examination of logs from GitLab, Jenkins, SonarQube, Nexus and DependencyTrack through individual Docker CLI sessions.
3. **Crossplatform heterogeneity:** Linux and Windows containers exhibit different runtime APIs, log collection mechanisms and performance metric exposure. Instrumentation patterns must accommodate platformspecific differences while maintaining unified telemetry schemas.

1.3.2 Organizational Problem

The organization experiences concrete operational difficulties stemming from limited observability. Recent incidents illustrate the problem severity:

- A GitLab database connection pool exhaustion incident in November 2025 was detected 3.5 hours after initial symptoms when multiple development teams reported failed pipeline executions. Root cause analysis required manual correlation of GitLab application logs, PostgreSQL container logs and Docker runtime inspection across two hosts.
- Intermittent Jenkins build failures in October 2025 attributed to insufficient executor capacity were diagnosed through trialanderror resource adjustments over multiple days. Historical resource utilization data was unavailable to inform capacity planning decisions.
- A disk space saturation event in September 2025 affecting the Nexus artifact repository caused cascading build failures. Detection occurred only after builds began failing with cryptic upload errors. The absence of proactive disk usage monitoring prevented preventive action.

These incidents share common characteristics: detection relies on user reports rather than automated alerting, diagnosis requires timeconsuming manual investigation across multiple systems without correlation tools, and preventive measures are hindered by lack of historical operational data. The organization estimates an average Mean Time To Detection (MTTD) of 2–4 hours and Mean Time To Recovery (MTTR) of 4–8 hours for infrastructure-related incidents. Development teams have reduced confidence in pipeline reliability, leading to defensive practices such as redundant build attempts and delayed integration of changes.

1.3.3 Scope and Boundaries

This research is explicitly scoped to nonprivileged containerized environments with mixed operating systems (Linux and Windows). The investigation examines containerfirst observability patterns and does not address scenarios where hostlevel access or privileged execution is available. The work focuses on CI/CD platform services (source control, build automation, static analysis) rather than generalpurpose application monitoring. Evaluation will be conducted within a single organizational context; generalizability to other constraint patterns and organizational settings remains to be established through future research.

1.4 Intended Contributions

The primary contributions planned for this project are:

- A containerfirst reference architecture specifying candidate patterns for collecting metrics, logs and traces from nonprivileged containers in mixed OS onpremise environments, documenting component placement, data flows and correlation mechanisms. The architecture will be evaluated against the constraints documented in this chapter.
- A reproducible prototype demonstrating feasible instrumentation patterns (sidecars, collectors, log forwarding) that operate without host administrative privileges, delivered as container images and deployment templates with configuration artifacts. The prototype will serve as a proofofconcept to assess practical viability.
- An empirical evaluation measuring changes in MTTD, MTTR, pipeline throughput and instrumentation overhead, including documented measurement procedures to enable reproducibility and independent verification. The evaluation will provide evidence to assess whether the approach meets operational requirements.
- Practical guidance and configuration artifacts to support potential adoption by operations teams constrained by privilege restrictions and inability to modify service images, subject to validation through practitioner feedback.

1.5 Research Objectives

The objectives guiding this project are:

- **O1 Reference architecture:** Design and evaluate a containerfirst reference architecture specifying exporters, collectors, storage backends and visualization components deployable without host administrative privileges, including component placement decisions and data flow patterns. The architecture will be assessed against the documented constraints to determine feasibility.
- **O2 Prototype implementation:** Develop a reproducible prototype consisting of container images and deployment templates that instrument representative CI/CD services in the target environment, with documented configuration procedures and deployment automation. The prototype will serve to validate the reference architecture in practice.
- **O3 Quantitative evaluation:** Design and execute an empirical evaluation comparing MTTD, MTTR, telemetry pipeline throughput and instrumentation overhead before

and after deployment. Document measurement procedures, baseline collection methods and fault injection protocols to enable reproducibility. The evaluation will assess whether measurable improvements can be achieved under the documented constraints.

- **O4 Performance constraints:** Characterize resource overhead through measurement of CPU, memory and network utilization. Document tradeoffs between telemetry granularity and resource consumption. The evaluation will assess whether instrumentation overhead can be maintained below 5% to support operational acceptability.
- **O5 Reproducibility and guidance:** Provide configuration templates, deployment scripts and documented procedures for constrained onpremise environments. Include decision criteria for tool selection and deployment pattern choices, subject to validation through practitioner feedback.

Each objective is defined with measurable outcomes and explicit deliverables. Objective O3 includes concrete measurement procedures (baseline collection, controlled fault injection, paired comparisons) to support quantitative assessment rather than qualitative description. Objective O4 establishes an explicit threshold (5% overhead target) to guide evaluation and inform adoption decisions; the research will assess whether this threshold can be achieved in practice.

1.6 Research Questions

This project addresses the following questions:

1. How can metrics, logs and distributed traces be collected from containerized CI/CD services when only nonprivileged container execution is permitted and host administrative access is unavailable?
2. Which instrumentation and deployment patterns (sidecars, agent collectors, push versus pull pipelines) enable effective crossserver correlation in mixed OS environments while maintaining acceptable resource overhead and operational complexity?
3. What practical constraints and tradeoffs (data fidelity, retention policies, network and CPU overhead, security boundaries) are introduced by nonprivileged telemetry collection, and how can these be quantified and mitigated within a reproducible reference architecture?

1.7 Methodology Overview

This project follows a design science research approach [1], combining systematic literature review, comparative technology assessment, artifact design, prototype implementation and empirical evaluation. Design science is appropriate for this investigation because it addresses a practical problem through the creation and evaluation of a novel artifact (the observability framework) in a specific organizational context. The research strategy integrates qualitative methods (literature synthesis, practitioner interviews) with quantitative methods (performance measurement, resource overhead characterization) to assess both technical feasibility and operational acceptability. The work is organized into five phases:

Phase 1 (Problem Analysis and Literature Review): A PRISMA-guided systematic review examines observability frameworks, containerized monitoring and CI/CD reliability literature to identify existing approaches, documented gaps and constraint patterns relevant to nonprivileged deployment contexts.

Phase 2 (Technology Assessment): Evaluation of candidate tools (including Prometheus [2], Loki, Jaeger, OpenTelemetry) against defined criteria: nonprivileged execution capability, crossplatform support (Linux and Windows), resource overhead characteristics and integration complexity. Assessment produces a justified tool selection with documented tradeoff analysis.

Phase 3 (Architecture Design): Specification of component placement (exporters, collectors, storage, visualization), data flow patterns and correlation mechanisms. Deliverables include deployment diagrams, configuration templates and documented design decisions with rationale.

Phase 4 (Prototype Implementation): Implementation and documentation of a reproducible prototype using selected instrumentation patterns (sidecars, collector containers). Deliverables include deployment automation scripts, configuration artifacts and operational documentation.

Phase 5 (Evaluation and Validation): Quantitative evaluation of MTTD, MTTR and instrumentation overhead through controlled fault injection experiments with documented baseline measurements. Qualitative validation through structured practitioner interviews captures operational acceptability and deployment constraints not reproducible in laboratory conditions.

The methodology balances artifact creation (reference architecture and prototype) with empirical validation (quantitative metrics and qualitative feedback). Quantitative evaluation establishes measurable outcomes; qualitative data addresses operational acceptability and realworld deployment considerations that quantitative metrics alone cannot capture.

1.8 Ethical Considerations

This research involves human subjects activities (structured practitioner interviews) and access to operational telemetry data. All activities will follow institutional ethical review procedures and approved protocols. Ethical clearance will be obtained before commencing interviews or collecting operational data. Participants will provide documented informed consent, and will be informed of their right to withdraw at any stage. Interview data and operational telemetry will be anonymized to remove identifying information (names, email addresses, specific host identifiers). Data handling will conform to institutional policies and applicable data protection regulations, including GDPR where relevant.

Production deployments will be preceded by staging environment validation and change management approval procedures to minimize operational disruption risk. The project observes data minimization principles: telemetry collection is limited to technical metrics necessary for evaluation objectives (CPU, memory, network usage, response times, error rates). Personal data collection, if required for specific evaluation activities, will be explicitly justified, documented and subject to institutional review board approval before proceeding.

When presenting empirical results in project deliverables and publications, host identifiers, network topology details and other sensitive configuration information will be anonymized

or generalized to protect organizational confidentiality. Access to production systems and telemetry data will be governed by organizational access control policies and limited to authorized project personnel. All collected data will be stored securely with appropriate access controls and will be retained only for the duration necessary to complete the research, after which it will be securely disposed of in accordance with institutional data retention policies.

1.9 Document Structure

This preparation report documents the research plan, including problem framing, literature review, and planned methodology. It does not present final results or conclusions; those will be developed during the thesis execution phase (January–June 2026) following the plan documented here. The remainder of this document is organized as follows: Chapter 2 presents the systematic literature review following Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) methodology, synthesizing existing observability research and identifying gaps relevant to constrained CI/CD environments; Chapter 3 describes project planning, methodology details, skill management, resource allocation and the execution timeline; Chapter 4 summarizes the work presented and outlines next steps toward prototype implementation and evaluation.

Chapter 2

Literature review

2.1 PRISMA Methodology Overview

A systematic literature review will be conducted following PRISMA guidelines to ensure comprehensive and reproducible coverage of the research domain. The search strategy encompasses the following elements:

Information sources:

- **Academic databases:** IEEE Xplore, ACM Digital Library, Springer, ScienceDirect, and Google Scholar for peer-reviewed publications
- **Technical repositories:** arXiv for preprints and working papers in systems and software engineering
- **Gray literature:** Technical documentation from established open-source observability projects, industry whitepapers, and practitioner reports from reputable sources (given the rapid evolution of container observability practices)

Search terms and queries: Core search queries will combine terms from three concept groups using Boolean operators:

1. *Observability concepts:* “observability”, “monitoring”, “telemetry”, “instrumentation”, “metrics”, “logging”, “tracing”, “distributed tracing”
2. *Container and deployment context:* “container”, “containerized”, “Docker”, “non-privileged”, “unprivileged”, “sidecar”, “agent”
3. *Application domain:* “CI/CD”, “continuous integration”, “continuous deployment”, “DevOps”, “reliability”, “incident response”, “MTTD”, “MTTR”

Example query structure: (*observability* OR *monitoring* OR *telemetry*) AND (*container* OR *containerized* OR *non-privileged*) AND (*CI/CD* OR *reliability* OR *incident response*)

Inclusion and exclusion criteria:

- **Inclusion:** Peer-reviewed publications from 2018 to 2025, technical reports and documentation addressing container telemetry collection, instrumentation patterns, performance evaluation, or reliability improvement in constrained or heterogeneous environments, comparative studies of observability approaches, and empirical evaluations of monitoring overhead
- **Exclusion:** Publications exclusively focused on privileged host access patterns without discussion of constraints, cloud-provider-specific solutions that cannot generalize to

on-premise infrastructure, purely theoretical work without practical implementation guidance, and papers predating 2018 (unless seminal works establishing foundational concepts)

Selection process: Literature selection will follow a three-stage process: (1) title and abstract screening to identify potentially relevant sources, (2) full-text review to assess relevance against inclusion/exclusion criteria, and (3) quality assessment focusing on methodological rigor, reproducibility, and applicability to constrained environments. Reference chaining (backward and forward citation tracking) will supplement database searches to identify additional relevant work.

Gray literature and industry sources: In addition to peer-reviewed academic literature, industry documentation (including technical whitepapers, platform documentation from established observability projects such as OpenTelemetry, Prometheus, and Grafana, as well as vendor technical reports) was identified through targeted web searches and documentation repositories. These sources were assessed for technical credibility, relevance to practitioner-oriented deployment patterns, and alignment with the operational constraints described above. Industry sources complement the empirical evidence base by documenting real-world instrumentation practices, configuration tradeoffs, and platform capabilities, though they were not subjected to the same formal quality assessment criteria applied to peer-reviewed publications. These materials inform the synthesis of practitioner-oriented approaches (Section 2.6) but are distinguished from the core empirical studies that form the primary evidence base.

2.1.1 Identification

The identification stage recorded search queries, databases queried, and the number of retrieved records. Queries were versioned and documented to enable replication of the search process.

2.1.2 Screening

Title and abstract screening was performed using documented inclusion/exclusion criteria; screening decisions and reasons for exclusion at this stage were recorded. When uncertain, items were advanced to fulltext review to avoid premature exclusion.

2.1.3 Eligibility

Fulltext review assessed whether each study met the inclusion criteria and whether it reported sufficient methodological detail for quality assessment. Studies that lacked sufficient detail were noted but excluded from quantitative synthesis.

2.1.4 Inclusion and Data Extraction

Data from included studies were extracted into structured evidence tables. Extraction fields included: study context, deployment environment (cloud/onpremise), privilege assumptions, telemetry modalities used, evaluation metrics and experimental design. Extraction was validated for accuracy and completeness.

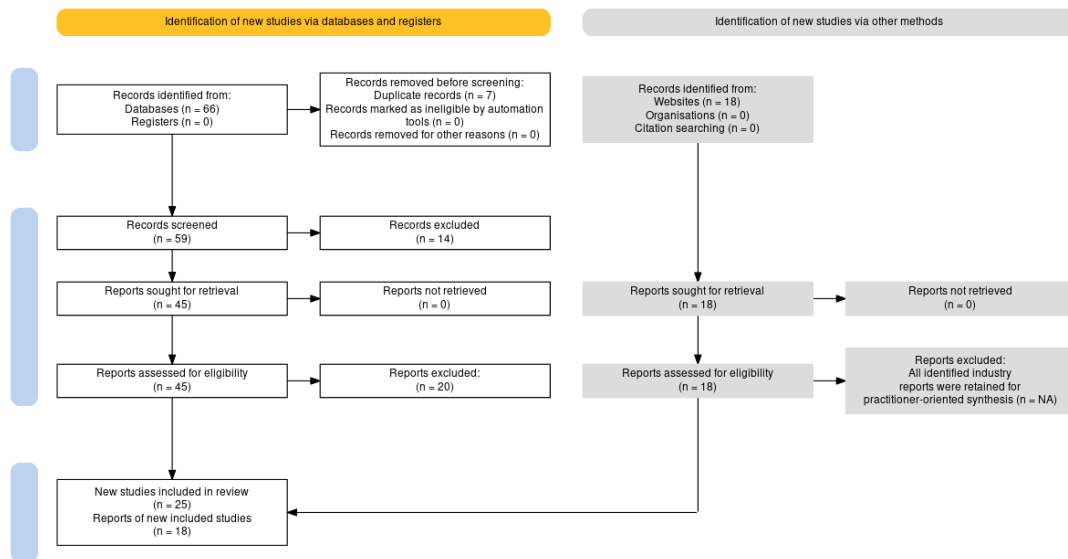


Figure 2.1: PRISMA flow diagram illustrating the systematic literature review process [3]. The diagram shows the progression from initial identification through screening, eligibility assessment, and final inclusion stages, documenting the number of records at each stage and reasons for exclusion.

2.2 PRISMA Flow Diagram

The systematic literature review process, as depicted in Figure 2.1, followed the four stages defined by the PRISMA 2020 protocol. The identification stage retrieved sixty-six records from academic databases (IEEE Xplore, ACM Digital Library, Springer, ScienceDirect, and Google Scholar). In parallel, eighteen additional records were identified through targeted searches of practitioner-oriented sources, including observability tooling documentation and CI/CD platform vendor materials. No records were identified from trial registers.

During deduplication and initial cleaning, seven records were removed due to duplication or because they did not constitute valid research publications (e.g., documentation links or malformed entries). The screening stage therefore evaluated fifty-nine records through title and abstract review against predefined inclusion and exclusion criteria focused on container observability, telemetry collection techniques, and relevance to constrained deployment environments. This screening resulted in the exclusion of fourteen records that were out of scope or lacked sufficient relevance.

The eligibility stage assessed forty-five reports through full-text review, examining methodological rigor, privilege assumptions, workload characteristics, and applicability to privilege-restricted CI/CD contexts. No reports were excluded due to retrieval issues. Following eligibility assessment, twenty-five peer-reviewed academic studies met all inclusion criteria and were retained as the core evidence base for the critical analysis presented in subsequent sections.

In addition to the academic literature, eighteen industry sources—including technical documentation from OpenTelemetry, Prometheus, Grafana, and CI/CD platform vendors such as GitLab and Jenkins—were retained as complementary evidence to document operational deployment patterns and platform capabilities. These practitioner-oriented sources were included as gray literature to contextualize real-world practices but were not subjected to the

same formal quality assessment criteria applied to peer-reviewed studies.

The twenty-five peer-reviewed academic studies that form the core evidence base of this systematic review are collectively cited here to ensure complete traceability between the PRISMA inclusion count and the bibliography [4]–[28]. While subsequent sections analyze a subset of these studies in detail—particularly those addressing instrumentation techniques, performance overhead, and deployment constraints—the remaining works inform background assumptions, contextual framing, and the identification of gaps relevant to constrained CI/CD observability.

2.3 Instrumentation and Deployment Assumptions in the Literature

The synthesis of included studies reveals that much of the documented observability practice and empirical evaluation assumes elevated privileges, homogeneous cloud environments, and a high degree of control over service code and deployment. This pattern is particularly evident in the use of sidecar architectures, Kubernetes operators, and kernel-level instrumentation frameworks. Such approaches, while effective in environments where orchestrators manage infrastructure, present significant barriers in heterogeneous or administratively restricted contexts where node access is controlled and third-party packaged services cannot be modified.

The comparative analysis of published observability systems demonstrates a consistent pattern of deployment and privilege assumptions that influence the applicability of proposed solutions to constrained environments. A substantial proportion of evaluated architectures presume host-level administrative access, either through privileged containers with direct socket access, kernel-level instrumentation frameworks, or cluster-wide operator deployments. Correira et al. [4] exemplify this pattern with a hybrid push-pull architecture requiring node-level agents with `cgroup` access for container metrics and RAPL interfaces for energy profiling, capabilities unavailable without elevated privileges. Similarly, Dinga et al. [5] deploy host-based collectors such as Metricbeat and cAdvisor with direct Docker API access, supplemented by application-level Zipkin tracing that necessitates explicit integration with containerized services. These deployment patterns, while effective in homogeneous cloud platforms where orchestrators manage infrastructure, present significant barriers in heterogeneous or restricted environments where node access is administratively controlled.

A second class of instrumentation approaches leverages kernel observability mechanisms, particularly extended Berkeley Packet Filter (eBPF), to achieve non-intrusive telemetry collection. Works such as Sharma et al. [22] and Kannan et al. [10] demonstrate that kernel-level probes can extract rich observability signals (including network flow metrics, system call traces, and container-level resource utilization) without requiring application code modifications. However, these mechanisms impose their own set of deployment prerequisites: Linux kernel versions that support stable eBPF interfaces (typically v5.15 or newer), capabilities such as `CAP_BPF` and `CAP_NET_ADMIN`, and deployment as privileged DaemonSets with node-wide scope. While such techniques offer substantial reductions in instrumentation overhead compared to sidecar proxies or per-service agents, they remain inapplicable in scenarios where host kernel access is restricted or where execution occurs on heterogeneous platforms without uniform kernel feature availability. The distinction between operational

feasibility and privilege requirements emerges as a critical dimension in assessing the transferability of observability architectures to environments lacking administrative control over underlying infrastructure.

Sidecar-based collection represents a third architectural family, evaluated extensively in service mesh contexts by Hasanth et al. [7] and Salcedo-Navarro et al. [20]. Sidecars intercept network traffic at the pod boundary, enabling transparent telemetry collection without modifying application logic. Yet sidecar deployments introduce resource multipliers proportional to service count, where each sidecar consumes CPU, memory, and network bandwidth, and rely on orchestrator-managed injection mechanisms that may not be available or permissible in restrictive CI/CD environments. Furthermore, latency analysis by Sahu et al. [19] indicates measurable microarchitectural contention between application and sidecar containers sharing vCPU resources, underscoring that deployment simplicity does not eliminate performance implications.

2.4 Performance Evaluation Practices and Reported Overheads

Empirical evaluations in the reviewed literature exhibit considerable heterogeneity in experimental design, workload selection, and performance metrics, complicating direct comparisons across studies. Benchmark applications vary from synthetic microservice suites such as Online Boutique [7] and DeathStarBench [27] to production-scale workloads deployed in commercial platforms [13]. Load generation techniques range from controlled concurrency ramps using tools such as Locust to stochastic arrival models with diverse traffic patterns. Overhead is quantified through disparate metrics, including CPU utilization percentages, absolute memory footprints, throughput degradation relative to baseline, and latency percentiles, reported at different granularities (per-node, per-container, system-wide aggregates). This methodological diversity reflects the absence of standardized observability benchmarking protocols for containerized systems, a gap that hinders reproducibility and cross-study synthesis.

Reported overhead magnitudes span a wide range depending on instrumentation technique and signal fidelity. Host-based eBPF solutions demonstrate notably lower resource consumption than alternative approaches: Sharma et al. [22] report CPU usage reductions of twenty-one to two-hundred-fourteen times compared to OpenTelemetry agents, sidecar proxies, and traditional host exporters. Kannan et al. [10] document approximately ten percent throughput degradation and point-five percent per-node CPU overhead for network flow monitoring via Traffic Control hooks, contrasting favorably with legacy packet capture tools that impose fifty to seventy percent throughput penalties. Conversely, full-stack distributed tracing through export-based OpenTelemetry collectors incurs substantial costs: performance evaluations on serverless and microservice workloads reveal throughput reductions ranging from nineteen to eighty percent and latency increases between seven and one-hundred-seventy-five percent [8]. Tail-based sampling strategies, while improving trace completeness, amplify collector CPU consumption by seventy-one percent relative to head-based sampling [11]. These figures underscore the existence of fundamental tradeoffs between signal completeness, collection latency, and resource overhead, tradeoffs that must be navigated in resource-constrained deployment contexts.

Several studies acknowledge methodological limitations that restrict the generalizability of

overhead assessments. Dinga et al. [5] note the inability to isolate container-level energy consumption using external power measurement, attributing observed increases to host system-wide effects rather than application-specific telemetry costs. Norgren [14] highlights that collector placement decisions—DaemonSet versus sidecar—interact with orchestrator resource limits and TraceContext propagation overhead in ways that are not easily predicted from component-level microbenchmarks. Ramachandran et al. [18] document latency amplifications between thirteen and forty-five times baseline when employing comprehensive eBPF syscall tracing for fault diagnosis, demonstrating that even low-overhead kernel mechanisms impose non-negligible penalties when probe invocation frequency is high. The variability of reported overheads reflects not only differences in experimental setup but also the sensitivity of performance outcomes to workload characteristics, instrumentation scope, and backend processing efficiency.

2.5 Discrepancies Between Research Environments and Constrained CI/CD Contexts

The operational assumptions underlying much of the published research diverge substantially from the constraints characteristic of on-premise CI/CD deployments in heterogeneous or administratively restricted environments. The majority of evaluated systems presume orchestrator-mediated infrastructure management, often Kubernetes, with the authority to deploy cluster-wide resources such as DaemonSets, CustomResourceDefinitions, and admission controllers. This model affords centralized configuration, uniform telemetry pipelines, and the ability to inject sidecars or agents into workloads declaratively. By contrast, CI/CD systems operating on shared infrastructure with limited user privileges cannot rely on such orchestrator capabilities. Executors may run as unprivileged containers without access to host namespaces, kernel interfaces, or Docker sockets, precluding the direct application of host-based or kernel-level instrumentation techniques documented in the literature.

Furthermore, research deployments typically assume homogeneous execution environments (identical kernel versions, uniform container runtimes, consistent network plugins) facilitating predictable instrumentation behavior. Real-world CI/CD infrastructure, particularly in organizations with diverse legacy systems or multi-cloud deployments, exhibits platform heterogeneity: varying operating system distributions, mixed kernel versions, and differing container runtime policies. Observability mechanisms dependent on specific kernel features, such as eBPF tracepoints introduced in recent Linux releases, fail silently or degrade gracefully depending on fallback support, introducing operational fragility. The assumption of code modifiability presents another divergence: academic prototypes often demonstrate instrumentation by integrating tracing libraries into purpose-built microservice applications, whereas CI/CD pipelines frequently execute third-party packaged software (build tools, testing frameworks, dependency resolvers) that cannot be recompiled or instrumented without violating support contracts or introducing version drift.

A third area of misalignment concerns the evaluation of correlation mechanisms across observability signals. Research systems implementing distributed tracing commonly propagate trace context through HTTP headers or gRPC metadata, mechanisms that require either application-level instrumentation or transparent proxy interception at service boundaries. In CI/CD contexts, correlation must often be reconstructed post hoc from logs, metrics, and execution metadata available through pipeline orchestration APIs, without the benefit of in-band trace propagation. This retrospective correlation problem (linking container resource

spikes to specific job identifiers, associating log error patterns with pipeline failure events) remains underexplored in the literature, which tends to assume instrumentation frameworks designed for long-lived service deployments rather than ephemeral task-based workloads. The temporal and architectural characteristics of CI/CD executions (short-lived containers, stateless task execution, batch job semantics) differ meaningfully from the microservice interaction patterns that dominate the observability research corpus.

2.6 Practitioner-Oriented Approaches and Industry Practices

Beyond the academic literature, industry documentation and technical reports from observability platform vendors provide complementary perspectives on telemetry collection in production environments. These sources emphasize operational pragmatism, configuration flexibility, and the incremental adoption of observability capabilities within existing infrastructure constraints. For instance, OpenTelemetry project documentation [29] outlines collector deployment patterns ranging from agent-per-host configurations to centralized gateway architectures, explicitly addressing tradeoffs between local buffering, network bandwidth consumption, and backend ingestion capacity. Prometheus operational guidelines [30] recommend pull-based metric collection with service discovery mechanisms that decouple telemetry exporters from backend topology, enabling declarative configuration suitable for dynamic containerized environments. Grafana Loki documentation [31] describes log aggregation without indexing full message content, reducing storage overhead at the cost of query expressiveness, a tradeoff relevant to resource-constrained deployments.

Vendor technical whitepapers and case studies, while less methodologically rigorous than peer-reviewed research, document deployment experiences across diverse organizational contexts. These reports frequently highlight challenges not emphasized in academic work: authentication and authorization models for multi-tenant telemetry pipelines, data retention policies balancing compliance requirements against storage costs, and observability tool sprawl arising from heterogeneous technology stacks. Datadog [32] and Dynatrace [33] technical blogs discuss the operational complexity of maintaining unified observability across hybrid cloud and on-premise infrastructure, noting that privilege limitations in customer-managed environments necessitate agent deployment models that sacrifice telemetry completeness for installation simplicity. Elastic observability architecture documentation [34] describes hybrid push-pull patterns where agents push metrics and logs to centralized endpoints, accommodating firewall traversal and egress-only network policies common in enterprise settings.

Industry practices also reveal the prevalence of logging-centric observability as a pragmatic compromise when comprehensive instrumentation is infeasible. Structured logging with correlation identifiers, combined with centralized log aggregation and pattern matching, provides limited but actionable visibility without requiring application code changes or privileged host access. Jenkins [35] and GitLab [36] CI/CD platform documentation emphasizes log-based troubleshooting workflows, reflecting the operational reality that pipeline executions are often diagnosed through console output rather than through distributed traces or granular metrics. While distributed tracing capabilities are available through plugins such as Jenkins OpenTelemetry [37], they are not enabled by default and are not central to documented debugging workflows. This logging-first approach aligns with the ephemeral nature of CI/CD workloads and the difficulty of instrumenting arbitrary third-party build tools. However, the literature provides minimal empirical guidance on the diagnostic effectiveness of log-only

observability compared to multi-signal approaches, leaving a gap between documented best practices and evidence-based performance validation.

2.7 Identified Gaps and Implications for Constrained CI/CD Observability

The synthesis of reviewed literature reveals several areas where existing research provides limited guidance for observability in privilege-constrained, heterogeneous CI/CD environments. First, empirical studies evaluating non-privileged telemetry collection patterns remain scarce. While sidecar-based approaches nominally operate without host-level privileges, performance evaluations typically occur in Kubernetes clusters where orchestrator-managed injection and resource allocation mitigate operational complexity. The feasibility and overhead characteristics of sidecar telemetry in environments where container orchestration is absent or restricted (such as standalone Docker executor environments or VM-based runners) have not been systematically characterized. Similarly, the applicability of application-level instrumentation libraries to third-party packaged software, a common scenario in CI/CD pipelines, receives minimal attention in the literature, which predominantly evaluates custom-built microservice applications amenable to source-level modification.

Second, the literature offers limited insight into post hoc correlation techniques suitable for ephemeral task-based workloads. Research on distributed tracing emphasizes in-band context propagation through service invocation chains, a model poorly suited to CI/CD executions where observability signals must be correlated retrospectively using pipeline metadata, container identifiers, and temporal alignment of logs and metrics. Techniques for reconstructing causal relationships between resource contention events and pipeline failures, or for attributing aggregate host metrics to individual containerized build tasks, are under-explored. This gap is significant because CI/CD observability fundamentally addresses a diagnostic problem (identifying which pipeline stage failed and why) rather than the performance optimization focus that dominates microservice observability research.

Third, the methodological practices documented in the literature provide insufficient guidance for evaluating observability interventions in controlled CI/CD settings. Standard microservice benchmarks such as Online Boutique or DeathStarBench model synchronous request-response patterns with stateless service replicas, characteristics that diverge from CI/CD workload semantics: batch job execution, stateful build caches, sequential pipeline stages with inter-stage dependencies. Metrics such as MTTD and MTTR, frequently cited as primary outcome measures for observability effectiveness, lack standardized operational definitions in the CI/CD context. Variability in how detection events are defined (alert trigger, operator awareness, automated remediation) and how recovery is measured (pipeline restart, job completion, service restoration) complicates the interpretation and comparison of reported improvements. The absence of reproducible benchmark artifacts (representative CI/CD workloads, fault injection scenarios, instrumentation templates) further hinders empirical validation of observability claims.

Finally, the literature exhibits limited engagement with operational constraints imposed by multi-tenant shared infrastructure, data residency requirements, and restricted network egress policies. Research prototypes typically assume unimpeded network connectivity to backend observability platforms, whereas enterprise CI/CD environments may mandate on-premise data retention, prohibit external telemetry export, or enforce egress filtering that complicates agent-to-collector communication. The implications of such constraints for

collector topology, buffering strategies, and telemetry completeness remain largely unaddressed. These gaps collectively indicate that while the observability research community has produced substantial evidence on instrumentation techniques, performance tradeoffs, and architectural patterns, the direct applicability of these findings to constrained CI/CD contexts requires further investigation and adaptation.

2.8 Mapping of Research Questions to Thesis Chapters

This section clarifies how the research questions defined in Chapter 1 are addressed across the dissertation structure. Chapter 2 provides the empirical and conceptual foundation by synthesizing existing observability research, identifying privilege assumptions and deployment constraints documented in the literature, and establishing where current evidence provides partial answers or reveals gaps. Subsequent chapters will build on this foundation to design, implement and evaluate solutions that directly answer the research questions within the constraints identified. The discussion below distinguishes between where questions are contextualized through literature synthesis (state of the art) and where they will be fully addressed through empirical artifact development and evaluation.

2.8.1 RQ1: Non-Privileged Telemetry Collection

The first research question asks: *How can metrics, logs and distributed traces be collected from containerized CI/CD services when only non-privileged containers are available (no host administrative access)?*

The systematic review presented in this chapter synthesizes existing instrumentation techniques—including sidecar exporters, agent collectors, kernel-level eBPF probes, and application-level tracing libraries—and documents their privilege requirements in detail (Section 2.3). The analysis reveals that most documented approaches in the literature presume host-level access, orchestrator-managed resource injection, or the ability to deploy privileged containers with elevated capabilities. For example, eBPF-based solutions [10], [22] require kernel access and capabilities such as `CAP_BPF`, while host-based exporters [4], [5] depend on Docker socket access or `cgroup` filesystem visibility. The gap analysis (Section 2.7) establishes that empirical studies evaluating non-privileged telemetry collection patterns specifically for CI/CD workloads remain scarce, and that sidecar evaluations predominantly occur in Kubernetes environments where orchestrator capabilities mitigate operational complexity.

Future chapters will address this research question through architectural specification of container-first exporters and collectors that operate without host privileges, prototype implementation demonstrating feasible instrumentation patterns for representative CI/CD services, and empirical evaluation of telemetry completeness, signal fidelity, and collection reliability under operational constraints. The evidence base established in this chapter informs the selection of candidate technologies and deployment patterns compatible with privilege restrictions documented throughout the literature.

2.8.2 RQ2: Cross-Server Correlation and Performance Trade-offs

The second research question asks: *Which instrumentation and deployment patterns (sidecars, agent collectors, push vs pull pipelines) best support cross-server correlation in mixed OS environments while maintaining acceptable resource overhead?*

Chapter 2 reviews sidecar architectures [7], [20], push-pull telemetry pipelines, and correlation mechanisms documented in microservice observability contexts (Sections 2.3 and 2.4). The synthesis reports empirically measured performance overheads from existing studies: sidecar proxies introduce measurable latency and microarchitectural contention [19], eBPF solutions demonstrate substantially lower CPU consumption compared to application-level instrumentation [22], and distributed tracing with full signal export can impose throughput reductions ranging from nineteen to eighty percent [8]. However, the literature provides limited guidance on heterogeneous OS environments (mixed Linux and Windows hosts) and post hoc correlation techniques suitable for ephemeral CI/CD workloads, where trace context propagation through HTTP headers or service mesh mechanisms may not be applicable (Section 2.5).

Future chapters will conduct comparative technology assessment to select deployment patterns that balance correlation effectiveness with resource constraints, implement topology and correlation mechanisms adapted to CI/CD execution semantics, and quantitatively evaluate correlation accuracy, telemetry pipeline throughput, and instrumentation overhead against the operational target of maintaining resource consumption below five percent where feasible. The performance baselines and trade-off characterizations documented in this chapter provide reference points for comparative evaluation.

2.8.3 RQ3: Constraints, Trade-offs and Mitigation Strategies

The third research question asks: *What are the practical constraints and trade-offs (data fidelity, retention, network and CPU overhead, security) introduced by non-privileged telemetry collection, and how can they be mitigated in a reproducible reference architecture?*

The literature synthesis documents privilege-dependent observability features—kernel probes requiring CAP_BPF, host exporters requiring Docker socket access, and orchestrator-managed sidecars requiring admission controller webhooks—and evaluates the deployment assumptions underpinning reviewed systems (Section 2.3). The analysis identifies that research prototypes typically presume capabilities unavailable in constrained environments: orchestrator-mediated infrastructure management, homogeneous execution platforms with uniform kernel versions, and the ability to modify or instrument application code (Section 2.5). The gap analysis (Section 2.7) highlights methodological limitations in existing evaluation practices, including the absence of standardized operational definitions for MTTD and MTTR in CI/CD contexts, limited guidance on data retention trade-offs under storage constraints, and the lack of reproducible benchmark artifacts suitable for controlled evaluation of constrained observability systems.

Future chapters will systematically document operational constraints encountered during architecture design, characterize trade-offs between telemetry completeness and resource overhead through prototype implementation, and provide configuration guidance addressing security boundaries, network egress restrictions, and multi-tenant isolation requirements. Empirical evaluation will quantify data fidelity (signal loss rates, sampling effects), retention feasibility (storage growth, query performance), and overhead characteristics (CPU, memory, network bandwidth) under representative CI/CD workloads. The reproducibility target includes deployment templates, measurement procedures, and benchmark scenarios that enable independent validation of reported findings. The constraint documentation and mitigation patterns established in this chapter inform the architectural decisions and evaluation methodology applied in subsequent work.

2.9 Summary and Implications for Subsequent Research

This chapter has established the methodological foundation for systematic literature review and synthesized existing research on container observability, with particular attention to deployment assumptions, performance characteristics, and the alignment between documented practices and the operational constraints of privilege-restricted CI/CD environments. The reviewed literature demonstrates that contemporary observability systems achieve rich telemetry collection through diverse instrumentation techniques (host-based kernel probes, sidecar proxies, application-level tracing libraries), each exhibiting distinct privilege requirements, performance overheads, and applicability boundaries. While kernel-level eBPF mechanisms offer substantial efficiency advantages, their dependence on host access and recent kernel features limits transferability to heterogeneous or restricted platforms. Sidecar patterns provide deployment flexibility but introduce resource multipliers and microarchitectural contention. Application-level instrumentation enables fine-grained observability at the cost of code modification and cross-platform compatibility challenges.

The comparative analysis reveals that much of the published research presumes operational capabilities unavailable in constrained CI/CD deployments: administrative host access, orchestrator-mediated resource management, and the ability to modify or instrument application code. Evaluation practices, while methodologically diverse, predominantly target long-lived microservice workloads rather than ephemeral task-based executions characteristic of CI/CD pipelines. Metrics such as MTTD and MTTR, though frequently invoked, lack standardized definitions suitable for comparing observability interventions across heterogeneous systems. These findings underscore the necessity of adapting existing instrumentation techniques and evaluation methodologies to the specific constraints and workload characteristics of on-premise, multi-tenant CI/CD infrastructure.

The mapping presented in Section 2.8 clarifies how the synthesis in this chapter informs the structure of subsequent work. Each research question is systematically grounded in the reviewed evidence: RQ1 (Section 2.8.1) establishes the privilege constraints documented across instrumentation approaches; RQ2 (Section 2.8.2) synthesizes performance trade-offs and identifies gaps in heterogeneous environment guidance; RQ3 (Section 2.8.3) documents operational constraints and methodological limitations in existing evaluation practices. The identified gaps (particularly the absence of empirical studies characterizing non-privileged telemetry collection in CI/CD contexts, limited guidance on post hoc correlation for ephemeral workloads, and methodological gaps in controlled evaluation frameworks) directly motivate the design, implementation and evaluation phases. Future chapters will leverage the documented privilege constraints and deployment patterns to specify a container-first reference architecture, implement a reproducible prototype that operationalizes feasible instrumentation approaches within identified boundaries, and evaluate the solution using standardized measurement procedures against the evidence base established here. Collectively, the evidence synthesized in this chapter establishes the research context and bridges the gap between general-purpose observability research and the specific requirements of constrained CI/CD observability.

Chapter 3

Project Planning and Methodology

This chapter presents how the dissertation project will be carried out, planned, monitored, and managed during the execution period (January–June 2026). Building on the problem framing in Chapter 1 and the state of the art in Chapter 2, the focus here is on the methodological approach, the planned artifacts and evaluation activities, the skills development plan (PREPD requirement), the project schedule, and risk management.

3.1 Objectives and Scope Alignment

The project objectives and scope defined in Chapter 1 remain aligned with the findings synthesized in Chapter 2. No scope changes are planned at this stage. Consequently, the work described below focuses on the execution strategy required to deliver the planned artifacts and produce defensible evidence for the dissertation.

3.2 Methodological Approach

The project is planned as an engineering-oriented research effort centered on the design, implementation, and evaluation of a container-first observability solution that can operate under the practical constraints identified in Chapter 1. The chosen approach is pragmatic and iterative: requirements and constraints inform successive design decisions, which are validated through a combination of controlled experimentation and practitioner-oriented feedback.

3.2.1 Planned Artifacts

The project will produce a set of artifacts intended to be directly auditable and reusable:

- **A1 – Reference architecture:** a documented architecture describing components, deployment topology, data flows, and correlation mechanisms suitable for containerized environments with limited privileges.
- **A2 – Prototype implementation:** a working prototype implementing the reference architecture in a staging environment, including configuration and operational procedures.
- **A3 – Automation and reproducibility assets:** scripts and configuration files to provision, deploy, and execute the evaluation campaign in a repeatable manner.
- **A4 – Evaluation protocol and dataset:** an experimental plan (scenarios, workloads, baselines, metrics) and the resulting collected data, with clear provenance.

- **A5 – Documentation package:** technical documentation for the architecture, deployment, and evaluation procedures, prepared to support both thesis writing and oral defense.

3.2.2 Development Process and Traceability

The development process is intended to ensure traceability from constraints to design decisions and from design decisions to evaluation evidence:

- **Constraint-to-design mapping:** constraints (e.g., limited privileges, heterogeneous execution environments) will be mapped to explicit architectural and implementation decisions.
- **Iterative refinement:** the architecture and prototype will be developed in short iterations, each concluding with a review of the produced artifact increment and the update of open issues.
- **Configuration management:** all artifacts will be maintained under version control, including deployment manifests, configuration, and evaluation scripts.
- **Acceptance criteria:** each major deliverable will be associated with objective acceptance criteria (e.g., successful deployment steps, reproducible execution of an experiment, completeness of documentation).

3.2.3 Planned Evaluation Strategy

Evaluation will be planned to produce evidence that the proposed solution is feasible under the stated constraints and that it provides measurable utility when compared with a defined baseline. The evaluation design will be bounded to what can be executed within the dissertation schedule and the available infrastructure.

Evaluation dimensions. The evaluation is intended to cover:

- **Feasibility under constraints:** whether the deployment and operation steps are compatible with the assumed privilege and platform limitations.
- **Operational overhead:** resource consumption and performance overhead introduced by the solution (measured using a defined workload and baseline).
- **Observability usefulness:** the extent to which the collected telemetry supports diagnosing representative failure and degradation scenarios.
- **Maintainability and operability:** the clarity of configuration, deployment, and troubleshooting procedures as captured in the documentation.

Experimental design (planning level). The experimental campaign will be structured as follows:

- **Baselines:** at least one baseline configuration will be defined to enable comparative measurements (e.g., no additional observability components, or minimal instrumentation).
- **Workloads and scenarios:** a small set of representative workloads and fault/degradation scenarios will be specified to exercise logging, metrics, and tracing paths.

- **Data collection:** procedures will be documented to collect resource metrics, request/-trace metadata, and logs, ensuring consistent time windows and environment configuration.
- **Analysis:** the analysis plan will define how measurements will be summarized and compared, and how qualitative observations will be recorded.

Practitioner feedback (planning level). Where access and availability permit, the project will include structured feedback from practitioners (e.g., hosting organization stakeholders) using an interview or review protocol. The protocol will define eligibility, consent, the question guide, and how responses will be summarized (e.g., thematic coding at a coarse granularity) to support defensible qualitative claims.

3.3 Skills Development Plan (PREPD)

This section defines a skills development plan aligned with the project phases. The plan identifies the competencies required for the dissertation, the current diagnosis, development actions, and objective assessment criteria.

3.3.1 Competency Diagnosis

Table 3.1: Competency Diagnosis and Targets

Competency	Current Level	Target Level	Relevance
C1: Observability engineering in containers	Intermediate: operational familiarity with containers and basic telemetry tooling	Advanced: design and deploy a constrained, production-oriented telemetry stack	Core competency to implement A1–A3
C2: Experimental evaluation and validity	Foundational: understanding of evaluation concepts; limited campaign execution	Intermediate: define base-lines, run experiments, analyze results, discuss threats	Required to produce A4 and defensible evidence
C3: Technical communication and stakeholder interaction	Intermediate: academic writing; limited structured practitioner engagement	Advanced: concise documentation, structured feedback collection, synthesis	Required for A5 and oral defense readiness

3.4 Project Planning and Schedule

The project plan is organized into phases with explicit deliverables and dependencies. The sequencing is designed to reduce risk by validating feasibility early (technology selection and staging deployment) before committing to extended evaluation work.

3.4.1 Phases, Activities, and Deliverables

3.4.2 Milestones and Dependencies

Milestones are defined to support periodic assessment and early detection of schedule risk. Key dependencies are: (i) technology selection before architecture freeze, (ii) staging deployment before extended evaluation, and (iii) availability of infrastructure and stakeholders before practitioner feedback.

Table 3.2: Skills Development Actions Mapped to Project Phases

Comp.	Planned Development Actions (by phase)	Assessment Criteria
C1	P1–P2: deep-dive on candidate tooling and deployment patterns; implement staging deployment; document operational procedures. P3: iterate architecture and prototype based on deployment feedback and constraint compliance.	Prototype deploys from documented procedure; architecture decisions are traceable to constraints; operational runbook is complete.
C2	P2: define evaluation protocol (metrics, baselines, scenarios, data collection). P3–P4: execute measurement campaign; perform comparative analysis; document threats to validity.	Evaluation protocol is reproducible; dataset provenance is documented; analysis includes baselines and validity discussion.
C3	P1: consolidate PREPD writing and structure. P3–P5: produce technical documentation; prepare interview/review protocol; synthesize practitioner feedback where applicable.	Documentation supports independent replication; feedback protocol is structured; writing and oral preparation are consistent and coherent.

Table 3.3: Planned Phases, Activities, and Deliverables (January–June 2026)

Phase	Period	Key Activities	Main Deliverables
P1	Jan	Project setup; consolidation of requirements and constraints; definition of acceptance criteria and logs (issues/risks).	Execution plan; project logs; refined acceptance criteria.
P2	Jan–Feb	Technology assessment and selection; proof-of-concept validations; definition of the architecture decision record structure.	Selection matrix and justification; initial architecture decisions.
P3	Feb–Mar	Reference architecture specification; prototype implementation; staging deployment; automation and reproducibility assets.	A1–A3 (architecture, prototype, automation); deployment runbook.
P4	Apr–May	Evaluation campaign execution: baseline definition, scenarios, measurements, data collection, preliminary analysis.	A4 (evaluation protocol and dataset); preliminary analysis notes.
P5	May–Jun	Consolidated analysis; practitioner feedback (when feasible); thesis writing and revision cycles; defense preparation.	A5 (documentation package); completed thesis manuscript.

Table 3.4: Planned Milestones and Acceptance Focus

ID	Planned Date	Milestone	Acceptance Focus
M1	End Jan	Technology selection completed	Selection rationale documented; fallbacks identified; feasibility validated via proof-of-concept.
M2	Mid Mar	Staging prototype deployed	Deployment is repeatable; core telemetry pipeline works end-to-end; runbook is usable.
M3	End Apr	Evaluation protocol frozen	Baselines and scenarios defined; collection procedures documented; validity considerations recorded.
M4	End May	Measurement campaign executed	Dataset collected with provenance; key analyses executed; issues and limitations captured.
M5	Mid Jun	Thesis ready for submission	Chapters integrated; evidence traceable to artifacts; revisions incorporated; defense preparation completed.

3.4.3 Gantt Chart

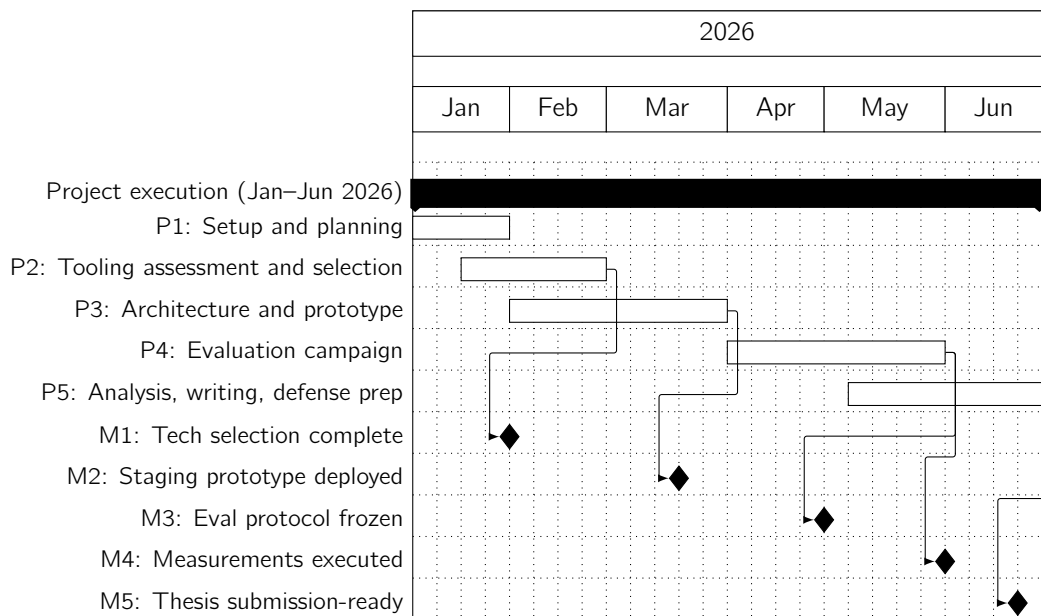


Figure 3.1: Project schedule overview (Gantt chart; January–June 2026)

3.5 Project Monitoring and Management

Project monitoring will be structured to ensure timely control of scope, schedule, and quality:

- **Governance and roles:** responsibilities will be split between the student (execution and reporting), the academic advisor (scientific guidance and quality control), and the hosting institution supervisor (infrastructure coordination and practitioner feedback facilitation when applicable).

- **Cadence:** meetings will be held regularly (e.g., bi-weekly with the academic advisor; weekly or as-needed with the hosting supervisor) to review progress against the plan and to unblock dependencies.
- **Progress tracking:** a lightweight project log will be maintained, including tasks, deliverables, open issues, decisions, and risk status.
- **Change control:** any scope adjustment will be documented with rationale, impact analysis (schedule/risk), and approval by the advisor.
- **Quality control:** each milestone will include an acceptance review focused on reproducibility, completeness of documentation, and alignment with constraints.

3.6 Risk Management

Risk management will be continuous and integrated with monitoring. Risks will be identified early, assessed by probability and impact, tracked through indicators, and mitigated through preventive actions. When mitigation is insufficient, a contingency plan will be activated to preserve the ability to complete the dissertation within the time constraints.

To keep the assessment consistent and auditable, this project uses a lightweight qualitative scale for both *Impact* and *Probability*. Impact captures the consequence to delivery (scope, schedule, or validity of results); probability captures the likelihood of occurrence within the January–June 2026 execution window. Monitoring indicators are defined as observable signals with thresholds (where feasible) that trigger mitigation or contingency actions.

Table 3.5: Risk Rating Scales and Measurement Approach

Dimension	Class	Operational definition (how it is measured)
Impact	H	Prevents delivering a core artifact (A1–A4) or forces a major scope reduction; or causes a schedule slip > 2 weeks on a milestone.
	M	Degrades evidence quality or requires rework; or causes a schedule slip of ~ 1–2 weeks.
	L	Localized issue with limited rework; schedule impact < 1 week.
Probability	H	Likely (> 60%): already observed signals or strong prior evidence in similar environments.
	M	Possible (20–60%): plausible given constraints; may occur depending on approvals/changes.
	L	Unlikely (< 20%): would require unusual conditions or multiple contributing events.

The risk register will be reviewed at least monthly and at each milestone review. For high-impact risks, mitigation status and contingency readiness will be explicitly reported in progress updates.

Table 3.6: Risk Register (technical, schedule, and external risks)

ID	Type	Risk Description	Impact	Prob.	Monitoring Indicators	Mitigation / Contingency
R1	Tech	Tooling or deployment does not comply with privilege/platform constraints, or is not reliable in the target environment.	H	M	Proof-of-concept fails to deploy >2 times; required capability missing; unstable telemetry pipeline.	Run early proof-of-concept; keep fall-back options; freeze a minimal viable architecture and reduce scope if needed.
R2	Tech	Telemetry pipeline quality and/or overhead is insufficient for the planned diagnosis scenarios.	H	M	Trace/log correlation breaks; missing identifiers; CPU/memory overhead exceeds agreed bounds; latency regression beyond baseline tolerance.	Validate correlation end-to-end early; tune configuration and sampling; prioritize fewer scenarios with stronger instrumentation; bound evaluation and document limitations.
R3	Schedule	Infrastructure access, deployment approvals, or environment instability delays evaluation activities.	H	M	Access not granted by planned date; blocked deployments; long coordination cycles; repeated environment outages.	Use staging as primary target; parallelize protocol design and writing; maintain schedule buffers; adjust campaign size to preserve M3–M5.
R4	Schedule	Writing and review iteration compresses due to technical delays, reducing dissertation quality.	H	M	Missed weekly writing checkpoint; advisor review windows missed; late integration of chapters/evidence.	Enforce weekly writing output; draft chapters incrementally; lock review dates; de-scope non-essential work to protect submission readiness.
R5	External	Limited stakeholder availability or ecosystem/policy changes reduce feedback feasibility or impose new constraints.	M	M	Low response rate; repeated rescheduling; new restrictions; deprecations impacting selected components.	Recruit early; allow asynchronous reviews; complement with advisor/expert review; choose conservative components; document assumptions and adapt plan.

Chapter 4

Conclusion and Next Steps

This chapter concludes the Preparation of Dissertation (PREPD) phase. The preparation work presented in this document establishes the foundation required to proceed into thesis execution. No results, findings, or experimental outcomes are presented in this document, as empirical work will be conducted during the subsequent dissertation phase.

4.1 Synthesis of Preparation Work

Chapter 1 framed the research problem: observability in containerized CI/CD platforms operating under nonprivileged access constraints and heterogeneous infrastructure. The problem arises from the operational gap between standard monitoring assumptions (host administrative access, privileged container execution, cloudmanaged platforms) and organizational realities in onpremise deployments where such capabilities are unavailable. The central research question examines how an integrated observability framework can be designed and evaluated to assess its potential to improve reliability and reduce incident response times under these constraints.

Chapter 2 synthesized the state of the art through systematic literature review following PRISMA guidelines. The review identified relevant observability patterns, instrumentation techniques, and evaluation approaches documented in prior work. The synthesis established that while observability frameworks for cloudnative environments are extensively documented, approaches specifically addressing nonprivileged containerized deployments in heterogeneous infrastructure remain underexplored in the literature.

Chapter 3 defined the project methodology and execution strategy. The approach centers on engineeringoriented research involving the design, implementation, and evaluation of a containerfirst observability solution. Planned artifacts include a reference architecture, prototype implementation, automation assets, evaluation protocol, and documentation package. The evaluation strategy specifies dimensions for assessing feasibility under constraints, operational overhead, observability usefulness, and maintainability.

4.2 Transition to Thesis Execution

The thesis execution phase will implement and evaluate the planned solution. The immediate next steps include:

- Complete technology selection based on compatibility with identified constraints, documented in a technology assessment matrix with explicit selection rationale.

- Design the reference architecture specifying components, deployment topology, data flows, and correlation mechanisms suitable for nonprivileged container environments.
- Implement the prototype in a staging environment, producing deployment templates, configuration artifacts, and operational procedures.
- Execute the evaluation campaign comparing baseline and instrumented deployments under defined workloads, including controlled fault injection scenarios.
- Collect and analyze measurement data, prepare dissertation chapters documenting implementation, evaluation findings, and implications for practice.

Validation targets defined in project planning include quantitative metrics (MTTD, MTTR), telemetry pipeline throughput verification, alert precision and detection coverage assessment via fault injection, and instrumentation overhead quantification with an operational target below 5% where feasible. These targets will be operationalized through specific measurement procedures and acceptance criteria documented in the evaluation protocol. Analysis will be restricted to tested contexts without overgeneralization.

The work documented in this PREPD confirms readiness to proceed with thesis execution. All preparatory activities required to initiate implementation and evaluation have been completed.

Bibliography

- [1] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [2] J. Volz, "Prometheus: Monitoring at soundcloud," *SoundCloud Developers Blog*, 2016.
- [3] N. R. Haddaway, M. J. Page, C. C. Pritchard, and L. A. McGuinness, *PRISMA2020: An R package and Shiny app for producing PRISMA 2020-compliant flow diagrams, with interactivity for optimised digital transparency and open synthesis*, version 1.1.1, 2022. doi: 10.5281/zenodo.5082518. [Online]. Available: <https://doi.org/10.5281/zenodo.5082518>.
- [4] M. Correia, W. Oliveira, and J. Cecílio, "Monintainer: An orchestration-independent extensible container-based monitoring solution for large clusters," *Journal of Systems Architecture*, vol. 145, p. 103035, Dec. 2023, issn: 13837621. doi: 10.1016/j.sysarc.2023.103035.
- [5] M. Dinga, I. Malavolta, L. Giamattei, A. Guerriero, and R. Pietrantuono, "An Empirical Evaluation of the Energy and Performance Overhead of Monitoring Tools on Docker-Based Systems," in *Service-Oriented Computing*, Cham: Springer Nature Switzerland, 2023, pp. 181–196. doi: 10.1007/978-3-031-48421-6_13.
- [6] M. Großmann and H. Klug, "Monitoring Container Services at the Network Edge," in *2017 29th International Teletraffic Congress (ITC 29)*, vol. 1, Sep. 2017, pp. 130–133. doi: 10.23919/ITC.2017.8064348.
- [7] K. M. Hasanth and D. Nadig, "Evaluating the Performance of Sidecar-based and Sidecarless Cloud-native Service Mesh Solutions," in *2024 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS)*, Dec. 2024, pp. 1–6. doi: 10.1109/ANTS63515.2024.10897788.
- [8] *Investigating Performance Overhead of Distributed Tracing in Microservices and Serverless Systems*, <https://dl.acm.org/doi/epdf/10.1145/3680256.3721316>. doi: 10.1145/3680256.3721316.
- [9] M. Janecek, N. Ezzati-Jivan, and S. V. Azhari, "Container Workload Characterization Through Host System Tracing," in *2021 IEEE International Conference on Cloud Engineering (IC2E)*, Oct. 2021, pp. 9–19. doi: 10.1109/IC2E52221.2021.00015.
- [10] P. G. Kannan, S. M. Gupta, D. Behl, E. Raichstein, and J. Takvorian, "Designing a lightweight network observability agent for cloud applications," in *Passive and Active Measurement*, Cham: Springer Nature Switzerland, 2024, pp. 262–276. doi: 10.1007/978-3-031-56249-5_11.
- [11] T. M. Karkan, "Performance Overhead Of OpenTelemetry Sampling Methods In A Cloud Infrastructure,"
- [12] J. Levin, "ViperProbe: Using eBPF Metrics to Improve Microservice Observability," 2020.
- [13] H. Liu, J. Zhang, H. Shan, *et al.*, "JCallGraph: Tracing Microservices in Very Large Scale Container Cloud Platforms," in *Cloud Computing – CLOUD 2019*, Cham: Springer International Publishing, 2019, pp. 287–302. doi: 10.1007/978-3-030-23502-4_20.

- [14] E. Norgren, "OPTIMIZING DISTRIBUTED TRACING OVERHEAD IN A CLOUD ENVIRONMENT WITH OPENTELEMETRY,"
- [15] K. K. Pappula, S. Anasuri, and G. P. Rusum, "Building observability into full-stack systems: Metrics that matter," *International Journal of Emerging Research in Engineering and Technology*, vol. 2, no. 4, pp. 48–58, 2021.
- [16] A. Pi, W. Chen, X. Zhou, and M. Ji, "Profiling distributed systems in lightweight virtualized environments with logs and resource metrics," in *Proceedings of the 27th International Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC '18, New York, NY, USA: Association for Computing Machinery, Jun. 2018, pp. 168–179. doi: 10.1145/3208040.3208044.
- [17] P. Raith, T. Rausch, P. Prüller, A. Furutanpey, and S. Dustdar, "An End-to-End Framework for Benchmarking Edge-Cloud Cluster Management Techniques," in *2022 IEEE International Conference on Cloud Engineering (IC2E)*, Sep. 2022, pp. 22–28. doi: 10.1109/IC2E55432.2022.00010.
- [18] G. S. Ramachandran, L. McDonald, and R. Jurdak, "FUSE: Fault Diagnosis and Suppression with eBPF for Microservices," in *Service-Oriented Computing*, Cham: Springer Nature Switzerland, 2023, pp. 243–257. doi: 10.1007/978-3-031-48421-6_17.
- [19] P. Sahu, L. Zheng, M. Bueso, S. Wei, N. J. Yadwadkar, and M. Tiwari, *Sidecars on the Central Lane: Impact of Network Proxies on Microservices*, Oct. 2023. doi: 10.48550/arXiv.2306.15792. eprint: 2306.15792 (cs).
- [20] A. Salcedo-Navarro, M. Garcia-Pineda, and J. Gutiérrez-Aguado, "K8sidecar: A Modular Kubernetes Chain of Sidecar Proxies for Microservices and Serverless Architectures," *Software: Practice and Experience*, vol. 55, no. 8, pp. 1305–1319, 2025, issn: 1097-024X. doi: 10.1002/spe.3423.
- [21] F. Sandberg, "Evaluating OpenTelemetry's Impact on Performance in Microservice Architectures,"
- [22] B. Sharma and D. Nadig, "eBPF-Enhanced Complete Observability Solution for Cloud-native Microservices," in *ICC 2024 - IEEE International Conference on Communications*, Jun. 2024, pp. 1980–1985. doi: 10.1109/ICC51166.2024.10622329.
- [23] B. Sharma, "IMPROVING MICROSERVICES OBSERVABILITY IN CLOUD-NATIVE INFRASTRUCTURE USING EBPF," Thesis, Purdue University Graduate School, Apr. 2023. doi: 10.25394/PGS.22682623.v1.
- [24] D. Soldani, P. Nahi, H. Bour, *et al.*, "eBPF: A New Approach to Cloud-Native Observability, Networking and Security for Current (5G) and Future Mobile Networks (6G and Beyond)," *IEEE Access*, vol. 11, pp. 57 174–57 202, 2023, issn: 2169-3536. doi: 10.1109/ACCESS.2023.3281480.
- [25] Y. Tan, S. Quan, Z. Zhu, and S. Gao, "Research on A Lightweight Service Mesh Architecture Using eBPF," in *2023 4th International Conference on Computer Engineering and Application (ICCEA)*, Apr. 2023, pp. 52–55. doi: 10.1109/ICCEA58433.2023.10135394.
- [26] T. Taylor, F. Araujo, and X. Shu, "Towards an Open Format for Scalable System Telemetry," in *2020 IEEE International Conference on Big Data (Big Data)*, Dec. 2020, pp. 1031–1040. doi: 10.1109/BigData50022.2020.9378294.
- [27] *TraceWeaver: Distributed Request Tracing for Microservices Without Application Modification*, <https://dl.acm.org/doi/epdf/10.1145/3651890.3672254>. doi: 10.1145/3651890.3672254.
- [28] M. Usman, S. Ferlin, A. Brunstrom, and J. Taheri, "DESK: Distributed Observability Framework for Edge-Based Containerized Microservices," in *2023 Joint European*

- Conference on Networks and Communications & 6G Summit (EuCNC/6G Summit)*, Jun. 2023, pp. 617–622. doi: 10.1109/EuCNC/6GSummit58263.2023.10188344.
- [29] OpenTelemetry Authors. “OpenTelemetry Collector: Architecture and Deployment Patterns.” Official documentation describing collector deployment patterns, configuration options, and operational considerations. (2024), [Online]. Available: <https://opentelemetry.io/docs/collector/> (visited on 12/15/2024).
 - [30] Prometheus Authors. “Prometheus: Operational Aspects.” Operational guidelines for pull-based metric collection, service discovery, and deployment patterns. (2024), [Online]. Available: <https://prometheus.io/docs/introduction/overview/> (visited on 12/15/2024).
 - [31] Grafana Labs. “Grafana Loki: Log Aggregation System.” Technical documentation describing log aggregation architecture, indexing strategies, and storage trade-offs. (2024), [Online]. Available: <https://grafana.com/docs/loki/latest/> (visited on 12/15/2024).
 - [32] Datadog. “Hybrid Cloud Monitoring: Best Practices.” Technical blog and whitepapers discussing agent deployment models and observability across hybrid infrastructure. (2023), [Online]. Available: <https://www.datadoghq.com/blog/> (visited on 12/15/2024).
 - [33] Dynatrace. “Enterprise Observability Architecture.” Technical documentation on agent deployment in customer-managed environments and multi-tenant observability. (2023), [Online]. Available: <https://docs.dynatrace.com/docs> (visited on 12/15/2024).
 - [34] Elastic. “Elastic Observability: Architecture and Deployment.” Documentation describing hybrid push-pull patterns, agent architecture, and enterprise deployment considerations. (2024), [Online]. Available: <https://www.elastic.co/guide/en/observability/current/index.html> (visited on 12/15/2024).
 - [35] Jenkins Project. “Jenkins: Monitoring and Observability.” Official documentation on pipeline observability, logging practices, and troubleshooting workflows. (2024), [Online]. Available: <https://www.jenkins.io/doc/book/system-administration/monitoring/> (visited on 12/15/2024).
 - [36] GitLab. “GitLab CI/CD: Observability and Monitoring.” Platform documentation describing log-based troubleshooting and pipeline monitoring capabilities. (2024), [Online]. Available: <https://docs.gitlab.com/ee/ci/> (visited on 12/15/2024).
 - [37] Jenkins Project. “Jenkins OpenTelemetry Plugin.” Plugin documentation for distributed tracing integration in Jenkins pipelines. (2024), [Online]. Available: <https://plugins.jenkins.io/opentelemetry/> (visited on 12/15/2024).